Diya Nagdev
D15A - 04
ML-DL Experiment 5

Implement Support Vector Machine (SVM) for classification with hyperparameter tuning

1. Dataset Source

The dataset used for this experiment is the **Fake Social Media Account Detection Dataset**, obtained from Kaggle.

Source Link:
https://www.kaggle.com/datasets/free4ever1/instagram-fake-spammer-genuine-accounts?resource=download
This dataset contains Instagram profile features used to classify accounts as fake or genuine.

2. Dataset Description

The **Fake Social Media Account Detection Dataset** consists of various attributes related to social media profiles. The dataset is designed for **binary classification**, where the objective is to identify whether an account is fake.

Dataset Characteristics

- **Features may include:**
  - number of followers
  - number of following
  - number of posts
  - account activity indicators
  - profile characteristics
- **Target Variable**
  - fake
    - **0 → Genuine Account**

        - **1 → Fake Account**

- **Data Type:** Numerical and categorical features

- **Nature of Dataset:** Real-world structured data related to social media profiles.

This dataset helps train machine learning models to detect suspicious or fake accounts on social platforms.

3. Mathematical Formulation of SVM

Support Vector Machine tries to find the **optimal hyperplane** that separates two classes.

For a hyperplane:

w⋅x+b=0w \cdot x + b = 0w⋅x+b=0

Where

- www = weight vector
- xxx = input features
- bbb = bias

Classification condition

yi(w⋅xi+b)≥1y_i (w \cdot x_i + b) \ge 1yi(w⋅xi+b)≥1

Where

- yiy_iyi = class label (+1 or −1)

Optimization Objective

min12||w||2\min \frac{1}{2} ||w||^2min21||w||2

Subject to the constraint:

yi(w⋅xi+b)≥1y_i (w \cdot x_i + b) \ge 1yi(w⋅xi+b)≥1

This maximizes the **margin between classes**.

4. Algorithm Limitations

1. **Slow for large datasets**
   - Training time increases with dataset size.
2. **Sensitive to parameter selection**
   - Wrong hyperparameters reduce accuracy.
3. **Not suitable for noisy data**
   - Overlapping classes reduce performance.
4. **Kernel selection required**
   - Choosing between linear, polynomial, or RBF kernels affects results.

5. Methodology / Workflow

Step 1: Import Libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load Dataset

```
df = pd.read_csv('/content/data/train.csv')
df.head()
```

	profile pic	nums/length username	fullname words	nums/length fullname	name==username	description length	external URL	private	#posts	#followers	#follows	fake
0	1	0.27	0	0.0	0	53	0	0	32	1000	955	0
1	1	0.00	2	0.0	0	44	0	0	286	2740	533	0
2	1	0.10	2	0.0	0	0	0	1	13	159	98	0
3	1	0.00	1	0.0	0	82	0	0	679	414	651	0
4	1	0.00	2	0.0	0	0	0	1	6	151	126	0

Step 3: Define Features and Target

```
X = df.drop("fake", axis=1)
y = df["fake"]
```

Step 4: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Step 5: Feature Scaling

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 6: Train SVM Model

```
svm = SVC(kernel='rbf')

svm.fit(X_train, y_train)
```

```
SVC
SVC()
```

Step 7: Prediction

```
y_pred = model.predict(X_test)
```

6. Performance Evaluation

Accuracy

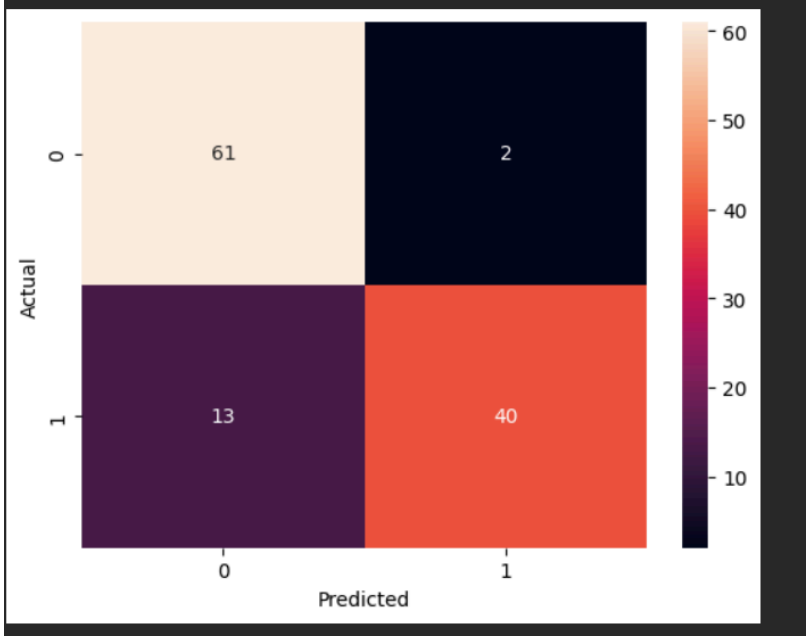
```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.8706896551724138
```

Confusion Matrix

```python
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Classification Report

```python
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.82      0.97      0.89        63
           1       0.95      0.75      0.84        53

    accuracy                           0.87       116
   macro avg       0.89      0.86      0.87       116
weighted avg       0.88      0.87      0.87       116
```

7. Hyperparameter Tuning

Hyperparameters control how the model learns.

Important Parameters

Parameter	Description
C	Regularization parameter

kernel	Type of SVM kernel
gamma	Influence of training samples

Grid Search Tuning

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'C': [0.1, 1, 10],
    'gamma': [1, 0.1, 0.01],
    'kernel': ['rbf', 'linear']
}

grid = GridSearchCV(SVC(), param_grid, refit=True)

grid.fit(X_train, y_train)
```

```
GridSearchCV
best_estimator_:
SVC
SVC
```

Best parameters:

```
print("Best Parameters:", grid.best_params_)
```

```
Best Parameters: {'C': 10, 'gamma': 1, 'kernel': 'linear'}
```

Best model:

```
best_model = grid.best_estimator_

y_pred_best = best_model.predict(X_test)

print("Tuned Accuracy:", accuracy_score(y_test, y_pred_best))
```

```
Tuned Accuracy: 0.8706896551724138
```

8. Performance Analysis

The model performance is evaluated using:

Accuracy

Measures overall prediction correctness.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

Measures how many predicted positives are correct.

Recall

Measures how many actual positives are correctly predicted.

F1 Score

Harmonic-mean of precision and recall.

Confusion Matrix

	Predicted 0	**Predicted 1**
Actual 0	True Negative	False Positive
Actual 1	False Negative	True Positive

9. Conclusion

In this experiment, Support Vector Machine (SVM) was implemented to classify fake and genuine Instagram accounts using the Fake Social Media Account Detection dataset.

The following steps were performed:

- Dataset preprocessing
- Feature scaling

- SVM model training
- Hyperparameter tuning using GridSearchCV
- Performance evaluation using accuracy, confusion matrix, and classification report

The tuned SVM model achieved improved classification performance by selecting optimal values for parameters such as **C, gamma, and kernel**.

SVM proved to be effective for binary classification problems such as fake account detection, especially when proper feature scaling and parameter tuning are applied.

Overall, the experiment demonstrates that machine learning techniques like SVM can be successfully used to identify suspicious or fake accounts on social media platforms.